

Code Lab 3

Burcu Erarslanoglu (5948355)

EE4C12: Machine Learning for Electrical Engineering Applications
Delft University of Technology

September 25, 2024

1 Task 1: Data Preparation

First, the data file containing the network traffic information is loaded into a Pandas data frame. The data has 6 features which are Receiving-data-rate-in-Kbytes-per-seconds, Transmitting-data-rate-in-Kbytes-per-seconds, Total-received-Kbytes, Total-Transmitted-Kbytes, Total-received-packets, Total-Transmitted-packets; and it has 1 classification label called as Class which assigns 4 categories to the types of usage. The data frame contains a total of 1350 samples. This data frame is then split into a feature data frame X and a label data frame y.

The next step in the data preparation process is calculating the correlation matrix of X. The correlation matrix along with its dimensions are shown below:

```
1 Receiving-data-rate-in-Kbytes-per-seconds \
2 Receiving-data-rate-in-Kbytes-per-seconds          1.000000
3 Transmitting-data-rate-in-Kbytes-per-seconds        0.748078
4 Total-received-Kbytes                               0.643117
5 Total-Transmitted-Kbytes                             0.591491
6 Total-received-packets                             0.636381
7 Total-Transmitted-packets                           0.496000
8
9 Transmitting-data-rate-in-Kbytes-per-seconds \
10 Receiving-data-rate-in-Kbytes-per-seconds          0.748078
11 Transmitting-data-rate-in-Kbytes-per-seconds        1.000000
12 Total-received-Kbytes                               0.681459
13 Total-Transmitted-Kbytes                             0.639431
14 Total-received-packets                             0.681218
15 Total-Transmitted-packets                           0.545820
16
17 Total-received-Kbytes \
18 Receiving-data-rate-in-Kbytes-per-seconds          0.643117
19 Transmitting-data-rate-in-Kbytes-per-seconds        0.681459
20 Total-received-Kbytes                               1.000000
21 Total-Transmitted-Kbytes                             0.935787
22 Total-received-packets                             0.981267
23 Total-Transmitted-packets                           0.820463
24
25 Total-Transmitted-Kbytes \
26 Receiving-data-rate-in-Kbytes-per-seconds          0.591491
27 Transmitting-data-rate-in-Kbytes-per-seconds        0.639431
28 Total-received-Kbytes                               0.935787
29 Total-Transmitted-Kbytes                             1.000000
30 Total-received-packets                             0.921450
31 Total-Transmitted-packets                           0.943146
32
33 Total-received-packets \
34 Receiving-data-rate-in-Kbytes-per-seconds          0.636381
```

```

35 Transmitting-data-rate-in-Kbytes-per-seconds 0.681218
36 Total-received-Kbytes 0.981267
37 Total-Transmitted-Kbytes 0.921450
38 Total-received-packets 1.000000
39 Total-Transmitted-packets 0.851075
40
41 Total-Transmitted-packets
42 Receiving-data-rate-in-Kbytes-per-seconds 0.496000
43 Transmitting-data-rate-in-Kbytes-per-seconds 0.545820
44 Total-received-Kbytes 0.820463
45 Total-Transmitted-Kbytes 0.943146
46 Total-received-packets 0.851075
47 Total-Transmitted-packets 1.000000
48 (6, 6)

```

As can be seen in line 48, the shape of the correlation matrix is 6×6 , therefore a square matrix is found as the correlation matrix as it should be.

Instead of all these entries that give the correlation matrix, the heatmap of the correlation matrix gives us information in a better-organized way. Below, this heatmap made using the Seaborn library is shown:

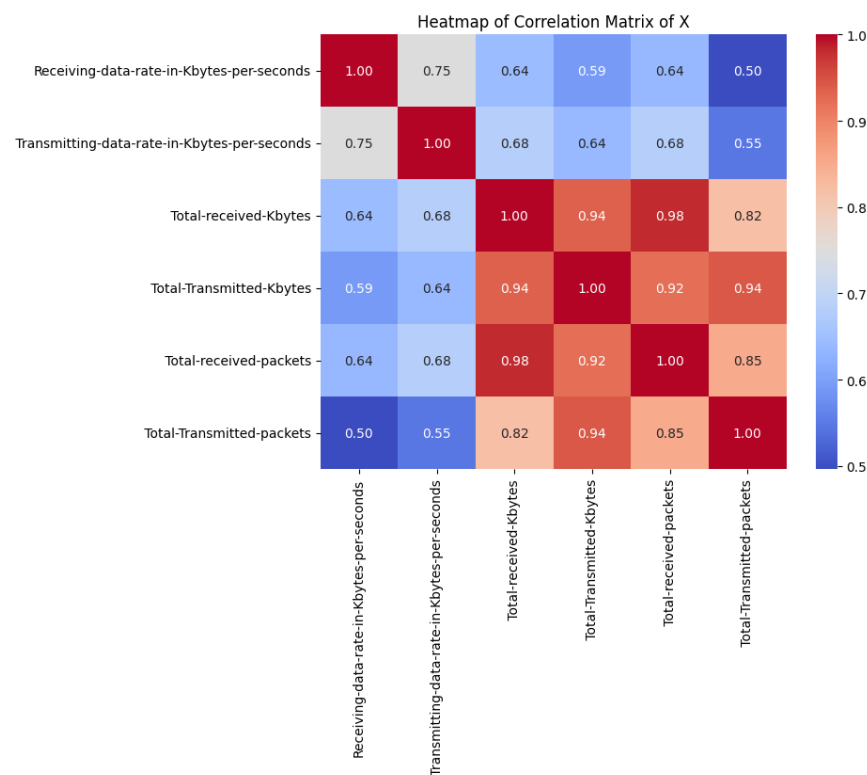


Figure 1: Heatmap of correlation matrix X.

The two variables that have the most correlation are then found. For this purpose, the diagonal is masked because the self-correlation between the two of the same features is always 1. This is not what we are interested in. After the masking, the two features that show the most correlation are found as follows:

```

1 (' Total-received-Kbytes', ' Total-received-packets')

```

The scatter plot of these most correlated features is given below:

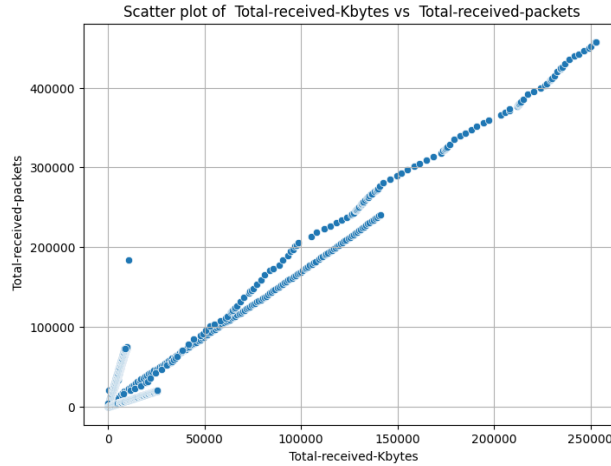


Figure 2: Scatter plot of Total-received-Kbytes vs Total-received-packets.

Next, an array is created named y_{2c} . The high-label samples are captured as 1 and the rest as 0. This is essentially binary classification. The ratio of high-class samples to total samples is given as:

```
1 Ratio of high-class samples to all samples: 0.4444444444444444
```

Moreover, another array named y_{mc} is created. This is the multiclass classification. The numerical labels are assigned to different classes as follows: 3-High, 2-Varied, 1-Low, 0-Buffer. The distribution of these labels in the data set is shown in the form of a bar plot as below:

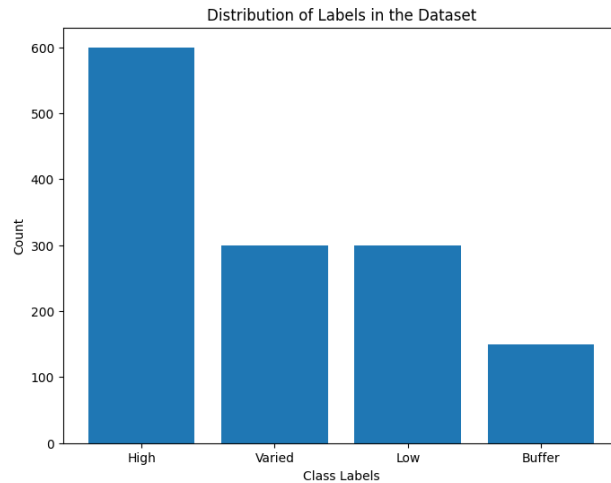


Figure 3: Distribution of labels in the dataset.

The next steps involve converting the data frames X , y_{2c} and y_{mc} into NumPy arrays. Also, the label vector elements are converted to integer values for y_{2c} and y_{mc} .

As the last step of the data preparation process, the feature matrix is scaled using the `StandardScaler()` function from Sklearn.

This concludes the coding done for Task 1, now the questions about this part can be answered.

Question 1

The variables with the highest correlation are Total-received-Kbytes and Total-received-packets. This correlation makes sense because the total number of received bytes is something that is related highly to the number of packets received. In networking, when you transmit more data in terms of Bytes, this requires you to send more packets. So, a high correlation between these features is expected.

Question 2

The ratio of high-class labels to all labels in binary classification was found as given below:

```
1 Ratio of high-class samples to all samples: 0.4444444444444444
```

Question 3

The shares of each class label in the multiclass classification are calculated as below:

```
1 Proportion of 'High' class (3): 0.4444
2 Proportion of 'Varied' class (2): 0.2222
3 Proportion of 'Low' class (1): 0.2222
4 Proportion of 'Buffer' class (0): 0.1111
```

These proportions can be visualized roughly from fig. 3 as well.

Question 4

In machine learning models, scaling improves the prediction capability because in many algorithms like the one studied in the previous code lab (support vector machines) input features are sensitive to scaling. If the features are on different scales, the model can give non-proportionate weights to certain features and this affects the prediction performance badly. When the data is scaled, all the features are normalized to the same scale. Typically, the data is transformed into standard normally distributed data (e.g. Gaussian with 0 mean and unit variance). As a result, the model can treat each feature equally.

2 Task 2: Binary classification with Logistic Regression

The binary classification that is used in this part detects whether the wireless network operates in high consumption mode or not. To start things off for this section, the data is first split into training and test sets with a test size of 0.25. Before the split, data is shuffled where the random state parameter is 4720. The two-class label vector of y_{2c} is used. In this part, the logistic regression model is used which is linear, to predict the probability of the sample belonging to a positive class.

A logistic regression model is first developed and then trained. The default parameters are used for this purpose. The trained model is then used to predict the test labels y_{2c} . With these predictions, the performance metrics of accuracy, recall (sensitivity), precision, and F1 score are found below:

```
1 Accuracy: 0.9378698224852071
2 F1 score: 0.9292929292929293
3 Recall score: 0.8679245283018868
4 Precision score: 1.0
```

The receiver operating characteristics (ROC) curve of training and test data is plotted in the same figure below:

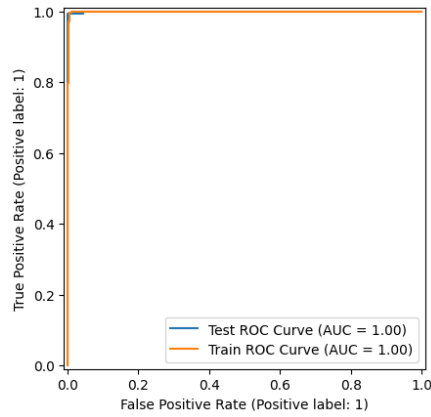


Figure 4: Receiver operating characteristic (ROC) curve of training and test data for logistic regression with the default parameters.

The confusion matrix of the logistic regression with the default parameters is plotted as below:

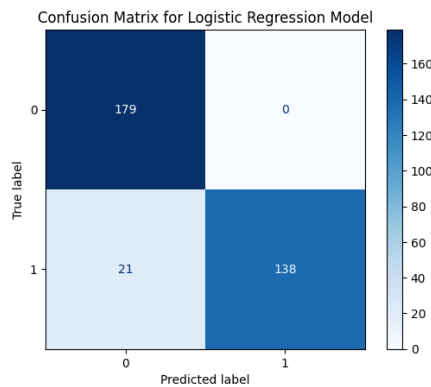


Figure 5: Confusion matrix for logistic regression with the default parameters.

For the second part of Task 2, we are instructed to adjust the class weight parameter inside the logistic regression function to reduce the number of missed high cases in other words the false negatives. A grid search-like approach is adopted to find the best weights for the two classes. The nested loop for the grid search includes an if statement that updates the lowest false negative scores along with the best weights for different weight combinations. The array of weights that I used for this part was $[0.01, 0.1, 0.5, 1, 2, 3, 5, 7.5, 10]$ and these resulted in the following best weights to reduce false negatives:

```
1 Best weight for class 0 (w_0): 0.01
2 Best weight for class 1 (w_1): 0.1
```

Some performance metrics about the logistic regression model with weights to decrease false negatives are given below:

```
1 Accuracy: 0.47041420118343197
2 F1 score: 0.6398390342052314
3 Recall score: 1.0
4 Precision score: 0.47041420118343197
```

The ROC curve of training and test data is plotted in the same figure below:

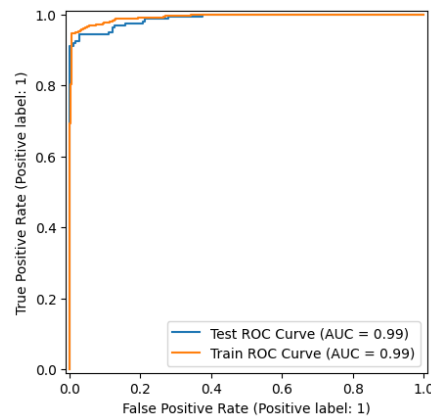


Figure 6: Receiver operating characteristic (ROC) curve of training and test data for logistic regression with modified weights.

The confusion matrix of the logistic regression with the default parameters is plotted as below:

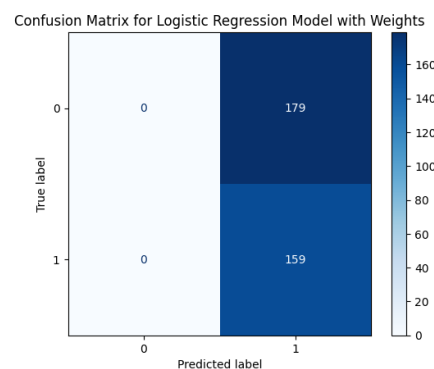


Figure 7: Confusion matrix for logistic regression with modified weights.

Question 1

As it was written before, the accuracy score for the logistic regression model was 0.937. This high accuracy score can be used to explain the near-perfect ROC curves for both the training and test data in fig. 4. The results in fig. 5 should be used to understand the ROC curves further. This confusion matrix shows that the model is very good at identifying class 0 with no errors; however for class 1, the model incorrectly predicts 21 instances. Moving on to the fig. 4, the ROC curve and AUC scores indicate a near-perfect performance for both the training and test sets. This means the model perfectly classifies positive (1) and negative (0) classes. We see that the training curve is slightly sharper thus with better performance than the test curve. Given that both the training and testing AUC scores are 1.0, there is a concern the model might be overfitting, potentially memorizing the data rather than learning general patterns to predict unseen data. Due to these results, along with the 21 mismatches, the results seem too good to be true for the ROC curve. When the test size was increased, for example, to 90% of the data, a noticeable decline in the test curve's performance was observed.

Question 2

The confusion matrix summarizes classification results by comparing the true labels and predicted labels that are produced by the classification model used. For the binary classification case, the confusion matrix is in the following format:

Table 1: Confusion Matrix for Binary Classification.

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

The places of the negatives and positives match the convention used in the plotted confusion matrices. The diagonal values of TN and TP show true predictions; whereas, FN and FP show misclassifications. The confusion matrix can be used for multi-class classification as well. In this case, the confusion matrix expands to represent each class. Again, the diagonal values are the correctly classified instances for each class and the off-diagonal values are misclassified instances. Overall, the confusion matrix details the model's performance, and allows us to understand the characteristics of the model more comprehensively than the accuracy metric.

Question 3

The weights used in logistic regression affect the classification results by applying a penalty for misclassifications of each class during model training. For our scenario, where we are trying to reduce the number of false negatives, the best weights were listed. We understand that a very low weight of 0.01 is attributed to class 0, so the importance of classifying instances of class 0 correctly is not so much. For class 1, weight is 0.1 which is small but higher than that of class 0. This means that the model is placing a bit more importance on correctly identifying class 1 instances which in the end reduces false negatives. When the confusion matrix for this case is observed in fig. 7, we see that reducing FN came with the cost of an increased number of false positives. This happens because the model becomes 10 times more sensitive to positive cases. This results in incorrect predictions of positives when the true labels are negatives. Finding a balance between FN and FP for a specific application would yield better results which are more balanced when the confusion matrix as a whole is considered.

Question 4

When comparing models, several metrics are important. Each of them offer different perspectives on model performance and guides us when we have to determine whether a model is good or not. Accuracy is a very common and simple metric. It may not tell us much about the cases of imbalanced data. In such scenarios, precision, recall, and F1-score come into play. For precision and recall, the outcome for a good classifier is close to 1. Precision is handy to evaluate the presence of FP and recall is useful to evaluate the presence of FN. The presence of FP of FNs steers the outcome far from the ideal classifier cases. The F1 score balances precision and recall, offering a single number that combines both, which is especially useful when you need to balance between FPs and FNs. Additionally there is the ROC curve, which provides a graphical representation of the trade-off between recall and the false positive rate across different thresholds, while the AUC summarizes the overall ranking ability of the model, giving a score between 0 and 1. With these being said, a good model depends on the specific problem, and for that specific problem, some metrics might be more useful than others. For example, there might be specific types of errors that you want to minimize. For balanced datasets, accuracy may suffice, but for imbalanced datasets other metrics should be found. The ROC curve offers a broader view on how the model performs but it might be subject to overfitting as we had in our case.

3 Task 3: Binary classification with Support Vector Machines

For this task, multiple support vector classifiers (SVC) are developed again for the binary classification problem.

3.1 Linear SVC with $[C=1.0, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$

First of all, the linear SVC is developed and trained with the parameters $[C=1.0, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$. The 4 performance metrics are:

```
1 Accuracy: 0.9733727810650887
2 F1 score: 0.970873786407767
3 Recall score: 0.9433962264150944
4 Precision score: 1.0
```

The ROC curve of training and test data is plotted in the same figure below:

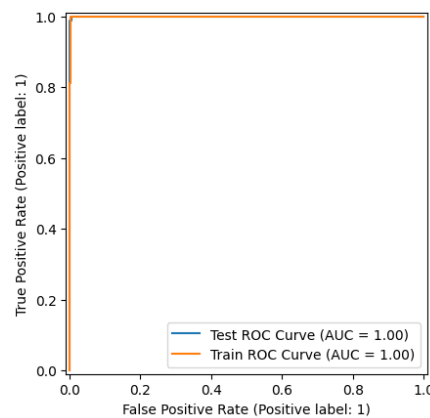


Figure 8: Receiver operating characteristic (ROC) curve of training and test data for linear SVM with $[C=1.0, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$.

The confusion matrix is provided below:

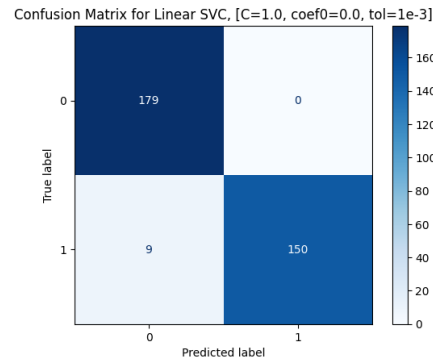


Figure 9: Confusion matrix for SVM with $[C=1.0, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$.

3.2 Linear SVC with $[C=0.0001, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$

The regularization term which adjusts the penalty factor for samples of training is decreased to 0.0001.

The 4 performance metrics are:

```
1 Accuracy: 0.5473372781065089
2 F1 score: 0.07272727272727272
3 Recall score: 0.03773584905660377
4 Precision score: 1.0
```

The ROC curve of training and test data is plotted in the same figure below:

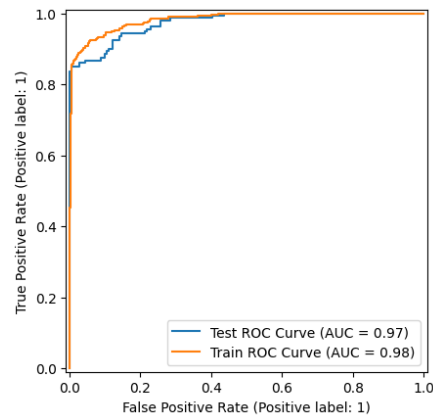


Figure 10: Receiver operating characteristic (ROC) curve of training and test data for linear SVM with $[C=0.0001, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$.

The confusion matrix is provided below:

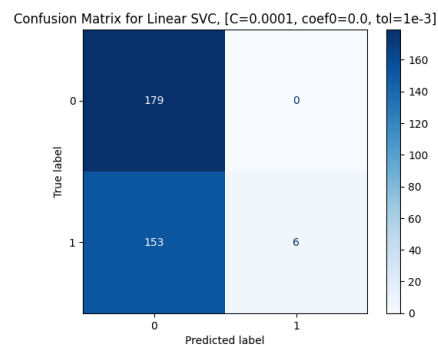


Figure 11: Confusion matrix for SVM with $[C=0.0001, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$.

3.3 Linear SVC with $[C=100, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$

The regularization term which adjusts the penalty factor for samples of training is increased to 100.

The 4 performance metrics are:

```
1 Accuracy: 0.985207100591716
2 F1 score: 0.9841269841269841
3 Recall score: 0.9748427672955975
4 Precision score: 0.9935897435897436
```

The ROC curve of training and test data is plotted in the same figure below:

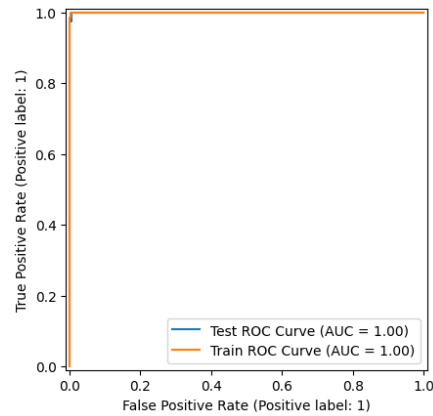


Figure 12: Receiver operating characteristic (ROC) curve of training and test data for linear SVM with $[C=100, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$.

The confusion matrix is provided below:

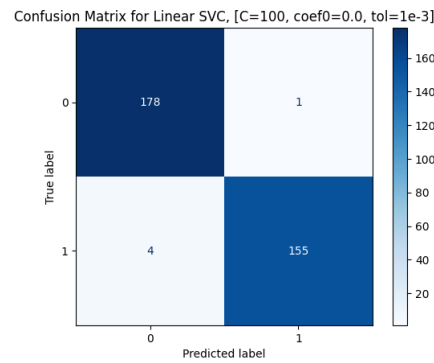


Figure 13: Confusion matrix for SVM with $[C=100, \text{coef0}=0.0, \text{tol}=1\text{e-}3]$.

3.4 Linear SVC with [C=1.0, coef0=0.0, tol=10]

The tunable hyperparameter of tolerance which determines the optimizer's stopping criteria is made 10 in this part.

The 4 performance metrics are:

```
1 Accuracy: 0.47041420118343197
2 F1 score: 0.6398390342052314
3 Recall score: 1.0
4 Precision score: 0.47041420118343197
```

The ROC curve of training and test data is plotted in the same figure below:

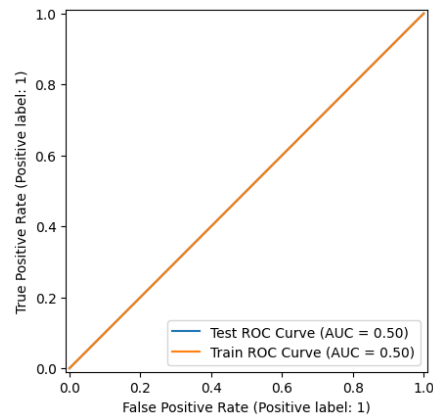


Figure 14: Receiver operating characteristic (ROC) curve of training and test data for linear SVM with [C=1.0, coef0=0.0, tol=10].

The confusion matrix is provided below:

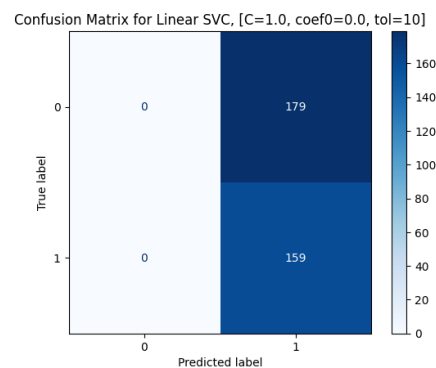


Figure 15: Confusion matrix for SVM with [C=1.0, coef0=0.0, tol=10].

3.5 Polynomial SVC of Degree 2 with $[C=1.0, \text{coef0}=0.0, \text{tol}=1e-3]$

The SVM classifier can use a polynomial kernel instead of a linear kernel. The 2nd degree polynomial SVC results are in this subsection.

The 4 performance metrics are:

```
1 Accuracy: 0.8757396449704142
2 F1 score: 0.8478260869565217
3 Recall score: 1.0
4 Precision score: 0.7358490566037735
```

The confusion matrix is provided below:

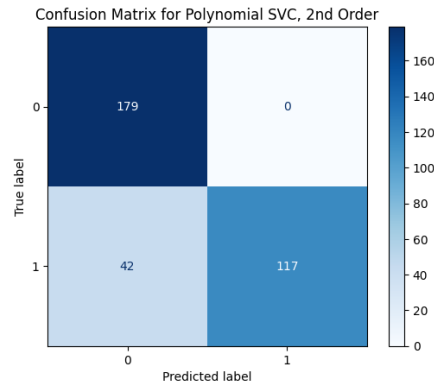


Figure 16: Confusion matrix for Polynomial SVM of 2nd Degree with $[C=1.0, \text{coef0}=0.0, \text{tol}=1e-3]$.

3.6 Polynomial SVC of Degree 3 with $[C=1.0, \text{coef0}=0.0, \text{tol}=1e-3]$

The 3rd degree polynomial SVC results are in this subsection.

The 4 performance metrics are:

```
1 Accuracy: 0.9349112426035503
2 F1 score: 0.9261744966442953
3 Recall score: 0.9928057553956835
4 Precision score: 0.8679245283018868
```

The confusion matrix is provided below:

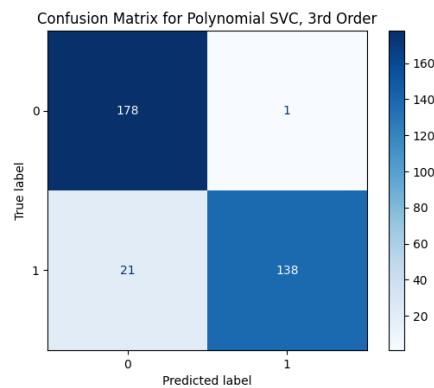


Figure 17: Confusion matrix for Polynomial SVM of 3rd Degree with $[C=1.0, \text{coef0}=0.0, \text{tol}=1e-3]$.

Question 1

Both linear SVM and logistic regression are linear classifiers that work well for binary classification problems. In the results, the linear SVM with $C=1$ had an accuracy of 0.973 and F1 score of 0.970; for logistic regression with the default parameters the performance was slightly poorer with an accuracy of 0.937 and F1 score of 0.929. The main difference between these two methods is their definition of the decision boundaries. Logistic regression outputs probabilistic predictions; whereas, SVM tries to find the optimal margin that maximally separates the classes. In this case, SVM slightly outperforms logistic regression in accuracy and F1 score, showing that it might be better at handling the dataset's decision boundary.

Question 2

The hyperparameter C for SVM controls the trade-off between maximizing the margin and minimizing classification errors. Therefore, the strength of the regularization is inversely proportional to C . A smaller C makes the margin larger but allows more misclassifications, while a larger C focuses on correctly classifying all training data, leading to a smaller margin. As seen from the results, when $C=0.0001$, the model performs poorly with an F1 score of 0.072, indicating underfitting, showing less than perfect classification characteristics for the ROC curve as well. However, as C increases to 100, the model achieves its best F1 score of 0.984, showing better classification with fewer errors. The ROC curve for this scenario shows excellent characteristics as well. This demonstrates that increasing C in general can lead to better performance.

Question 3

The tolerance parameter in SVM sets the stopping criterion for the optimizer in other words it determines when the model's optimization process should stop. When the tolerance is set to a high value like 10, the model converges quickly but potentially prematurely which leads to suboptimal results. As it can be observed from the metrics, when tolerance was changed to 10, the accuracy dropped to 0.470 and the F1 score to 0.639. This suggests that the model stopped too soon before an optimal decision boundary was reached. A lower tolerance would allow the optimizer to continue refining the boundary which will in result lead to a better performance.

Question 4

To answer this question, we need to know the relationship between features. If they are complex and potentially nonlinear, a polynomial kernel could be better. In this problem, the linear SVM with $C=100$ has the highest F1 score of 0.984, while the polynomial kernel of degree 3 achieves an F1 score of 0.926. So the linear kernel performs better here. When the data is not linearly spread, a nonlinear kernel like a polynomial kernel can model the complex decision boundaries much better which leads to a better performance overall. However, since the linear model performs the best here for SVM, we can say that the data is probably linearly separable.

Question 5

Based on the F1 scores, the models can be ranked from highest to lowest performance as follows:

- (1) Linear SVM with $C=100$ (F1 score = 0.984),
- (2) Linear SVM with $C=1$ (F1 score = 0.970),
- (3) Polynomial SVM with Degree 3 (F1 score = 0.926),
- (4) Polynomial SVM with Degree 2 (F1 score = 0.847),
- (5) Linear SVM with $\text{tol}=10$ (F1 score = 0.639), and finally,
- (6) Linear SVM with $C=0.0001$ (F1 score = 0.073).

This ranking highlights the importance of tuning hyperparameters such as C and tolerance; as well as the kernel choice in order to maximize model performance.

4 Task 4: Multi-class classification

This task deals with the multi-class classification that predicts the best among a set of labels. In our case, we have 4 labels: 3-High, 2-Variied, 1-Low, and 0-Buffer. This section considers logistic regression and SVM. For logistic regression 2 different approaches are studied which are the One-vs-All approach and the multi-class approach. To start things off for this section, the data is first split into training and test sets with a test size of 0.25. Before the split, data is shuffled where the random state parameter is 4720.

4.1 Logistic Regression

4.1.1 One-vs-All Approach

For this approach, binary classifiers are constructed for each of the 4 classes. As stated in the assignment description, the final prediction is based on the model with the highest score or the largest (signed) distance from the classification boundary. The multi-class classification for the one-vs-all approach is ensured via `multi_class = 'ovr'` parameter inside the `LogisticRegression` function. The multi-class label vector y_{mc} is used.

The accuracy of the model is given below:

```
1 Accuracy: 0.8698224852071006
```

The generated confusion matrix is given below:

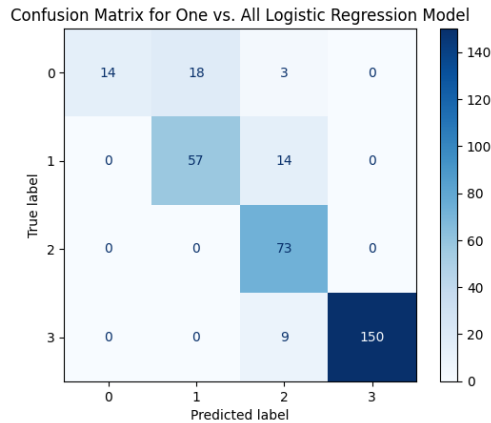


Figure 18: Confusion Matrix for One-vs-All Logistic Regression Model.

4.1.2 Multi-Class Approach

For this approach, the multi-class classifiers are trained by using the multi-class cross-entropy loss. The multi-class classification for the multi-class approach is ensured via `multi_class = 'multinomial'` parameter inside the `LogisticRegression` function.

The accuracy of the model is given below:

```
1 Accuracy: 0.9171597633136095
```

The generated confusion matrix is given below:

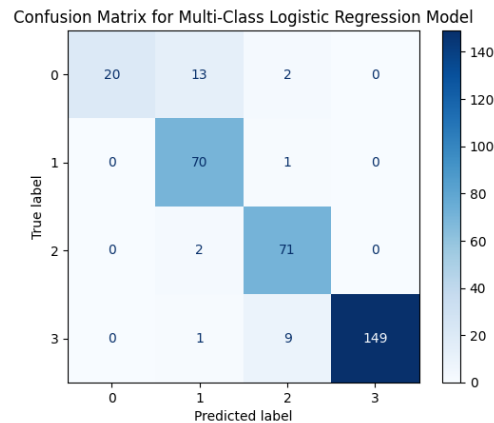


Figure 19: Confusion Matrix for Multi-Class Logistic Regression Model.

4.2 Support Vector Machine

In addition to logistic regression, SVM can also be used relying on the one-vs-all approach. Linear SVC is constructed as a one-vs-all classifier with the parameters of $[C=1.0, \text{coef0}=0.0, \text{tol}=1e-3]$.

The accuracy of the model is given below:

```
1 Accuracy: 0.9497041420118343
```

The generated confusion matrix is given below:

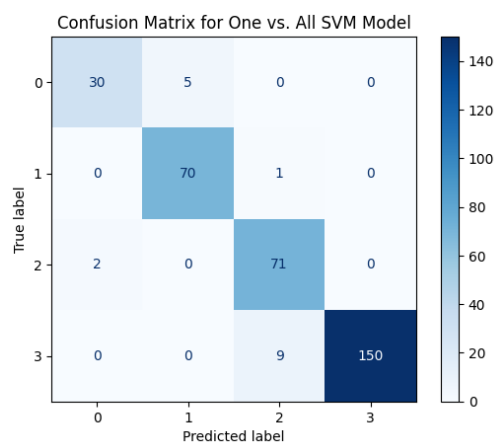


Figure 20: Confusion Matrix for One-vs-All SVM Model.

Question 1

For logistic regression, two different approaches were considered. The one-vs-all approach had an accuracy of 0.8698, whereas the multi-class approach involving the multinomial cross-entropy loss had a higher accuracy of 0.9172. We can see an improvement in performance for the multi-class approach because this one optimizes the predictions using cross-entropy loss. Such an approach is much more suitable when dealing with more than two classes because in the one-vs-all approach splitting the task into separate binary classifiers to handle multiple classes isn't very effective. As we have four classes, this approach is more optimal and yields results with higher accuracy. When fig. 18 and fig. 19 are compared, for example for class 1, we see a significant decrease in the mismatches when the multi-class approach is used.

Question 2

In between the methods of logistic regression and SVM, SVM is more demanding computationally than logistic regression because it requires the computation of support vectors and then solving quadratic programming problems. When a large dataset with multiple classes is used, this process becomes computationally demanding. Whereas, in the case of logistic regression, simpler convex optimization problems are solved to contain the output into a Sigmoid (logistic) function.

Question 3

In Tasks 2 and 3, we were able to use metrics like precision, recall, and F1-score because they are defined for binary classification tasks. To apply them in a multi-class classification task, they need to be adapted. However, the accuracy metric can be used directly. For precision, recall, and F1-score, these metrics must be calculated for each class, typically using a one-vs-rest approach, where each class is treated as the positive class at different times. This allows the multi-class structure to be handled properly and as a result, produces results for metrics other than accuracy.

Question 4

Logistic regression can directly handle multi-class classification using the multinomial parameter. This is an extension of the logistic regression for multi-class classification and it is formally called multinomial logistic regression. In contrast, SVM does not support multi-class classification because it was originally designed for binary classification. However, there are algorithms that reduce the multi-class task to several binary tasks. Some algorithms include one-vs-one and one-vs-rest approaches which are designed to handle multiple classes with the SVM.

Question 5

For the one-vs-all logistic regression, the accuracy was the lowest at 0.8698. For classes 0 and 1, the mismatch is quite high however for class 2 there are no mismatches. For the multi-class logistic regression, the accuracy has an improvement with 0.9172. It performs better overall, especially with fewer mismatches in classes 1 and 2; however, 2 mismatches are introduced for class 2. For the SVM with the one-vs-all approach, the accuracy is highest with 0.9497. It performs best among all models, with fewer mismatches, especially in class 0. For the other classes, the results are quite similar to the multi-class logistic regression.

Question 6

For the one-vs-all logistic regression, class 0 (Buffer) has higher mismatches, with 18 samples misclassified as class 1 and 3 samples misclassified as class 2. For the classification of class 1 and class 3, there are 14 and 9 mismatches respectively. For the multi-class logistic regression, class 0 (Buffer) again has slightly more mismatches compared to other classes with 15 mismatches. For class 1 there is 1, for class 2 there are 2 and for class 3 there are 10 mismatches.

Question 7

For the one-vs-all SVM, class 3 (High) has the highest mismatch with a total of 9 mismatches. The mismatches are to class 2. For class 2, 2 samples are misclassified as class 0; for class 1, 1 sample is misclassified as class 2, and for class 0, 5 samples are misclassified as class 1. However, the overall mismatch rate is lower compared to logistic regression, especially for class 0.

5 Bonus Task

The result for the bonus task is below:

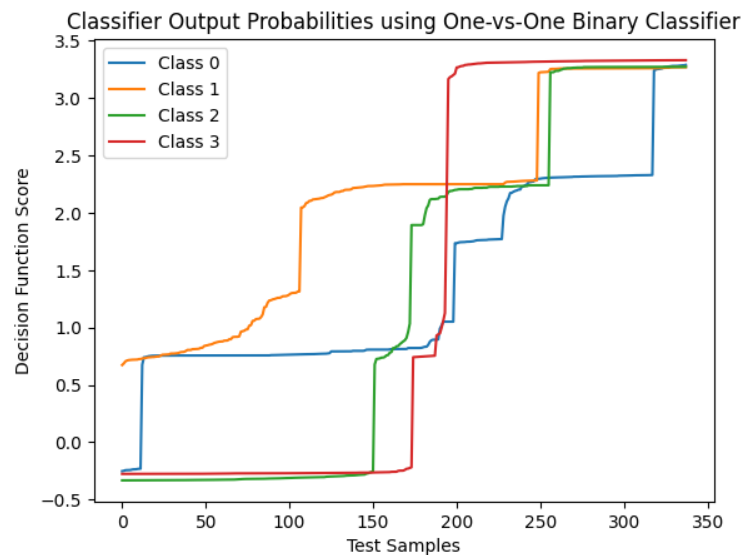


Figure 21: Classifier Output Probabilities using One-vs-One Binary Classifier for Multi-Class Classification

To achieve multi-class classification, by using only binary classifiers I chose the `OneVsOneClassifier` from `sklearn`. After this choice, a train-test split is performed as usual and for this purpose I used the same parameters that were used in other tasks of the code lab. Then, one-vs-one classification was carried out via `OneVsOneClassifier`. This is a binary classifier which is trained for each possible pair of classes. I used logistic regression as the base classifier within this strategy. After training, the model predicts the decision function scores for the test data using `decision_function`. As the final step, the decision function scores in other words the probabilities for each class are sorted in ascending order to analyze the predictions for each class.

These steps work effectively because it reduces the multi-class problem to multiple binary classification problems. Meaning, each binary classifier distinguishes between two classes at a time. For the predictions, all binary classifiers vote for their preferred class, and the class with the most votes is assigned to the sample.